Name: Praisel Ekpenyong (2025/A/CYB/0205)
Email: p.ekpenyong8162@miva.edu.ng
Department: Cybersecurity
Matric Number: 2025/A/CYB/0205

COS202 - Computer Programming II - Lab Assessment

Part 1: Source Files/Code

The complete C++ source is hosted on GitHub
https://github.com/praisel-ekpenyong/COS202_SCAPMS

i. Header Files

- **model/PatientRecord.h** — Base patient class.
- **model/GeneralPatient.h** — General outpatient class.
- **model/EmergencyPatient.h** — Emergency triage patient class.
- **model/FollowUpPatient.h** — Follow-up patient class.
- **model/Appointable.h** — Appointment scheduling interface.
- **model/Appointment.h** — Appointment data class.
- **model/AppointmentStatus.h** — Appointment status Enum.
- **model/PatientCategory.h** — Patient category Enum.
- **model/ClinicDatabase.h** — In-memory clinic database and cache.
- **controller/ClinicManager.h** — Core clinic workflow manager.
- **controller/SearchEngine.h** — Linear, binary, and recursive search.
- **controller/PatientSorter.h** — Selection sort and merge sort.
- **controller/SortAlgorithms.h** — Template sorting implementations.
- **controller/ReportEngine.h** — Clinic reporting and text export.
- **controller/UndoableAction.h** — Administrative undo action structures.
- **utils/Validators.h** — Input validation utilities.
- **utils/IDGenerator.h** — Auto-incrementing ID generator.
- **utils/FileHandler.h** — CSV file saving and loading.
- **utils/Exceptions.h** — Custom exception classes.
- **utils/AppInfo.h** — Application metadata definitions.

ii. Source Files

- **model/PatientRecord.cpp** — Base patient implementation.
- **model/GeneralPatient.cpp** — General patient implementation.
- **model/EmergencyPatient.cpp** — Emergency patient implementation.
- **model/FollowUpPatient.cpp** — Follow-up patient implementation.
- **model/Appointment.cpp** — Appointment implementation.
- **model/ClinicDatabase.cpp** — Database storage and cache implementation.
- **controller/ClinicManager.cpp** — Clinic manager logic and undo stack.
- **controller/SearchEngine.cpp** — Search algorithms implementation.
- **controller/PatientSorter.cpp** — Sorting algorithms implementation.
- **controller/ReportEngine.cpp** — Report generation logic.

- controller/UndoableAction.cpp — Undoable action implementations.
- utils/Validators.cpp — Form validation rules implementation.
- utils/IDGenerator.cpp — ID persistence implementation.
- utils/FileHandler.cpp — CSV reading and writing implementation.
- utils/AppInfo.cpp — Application metadata implementation.

iii. GUI Files and Resources

- gui/MainWindow.h / MainWindow.cpp — Main window and tab navigation.
- gui/DashboardPanel.h / DashboardPanel.cpp — Overview dashboard and statistics.
- gui/PatientPanel.h / PatientPanel.cpp — Patient registration and table.
- gui/AppointmentPanel.h / AppointmentPanel.cpp — Appointment scheduling and filters.
- gui/QueuePanel.h / QueuePanel.cpp — Priority triage waiting queue.
- gui/SearchPanel.h / SearchPanel.cpp — Record searching and sorting panel.
- gui/ReportPanel.h / ReportPanel.cpp — Clinic reports viewing and export.
- gui/AdminPanel.h / AdminPanel.cpp — Administrative undo and action log.
- gui/Theme.h / Theme.cpp — Application styling and color palette.
- gui/main.cpp — Application entry point.
- gui/resources.qrc — Application icon resource bundle.

iv. Data Files Required to Run the Application

- data/patients.csv — Sample patient records.
- data/appointments.csv — Sample appointment records.
- data/counters.csv — Persistent ID counter file.

Build and run instruction

```
# Configure project
cmake -B build -G Ninja -DCMAKE_BUILD_TYPE=Release

# Build application binary
cmake --build build

# Launch application
./build/gui/SCAPMS.exe
```

Part 2: Project Report

i. Project Description

The Smart Clinic Appointment & Patient Management System (SCAPMS) is a desktop application written in C++17 and Qt 6 Widgets. It manages patient records, book appointments, run waiting queues. Sort and search data, export simple reports and undo mistakes or actions. All records are saved to CSV files. The app also loads a set of sample clinic records when it starts up, so you can test every screen right away.

The code is organized into four namespaces:

- **scapms::model:** handles patients, appointments, and database storage.
- **scapms::controller:** handles the core logic, search, sort, reports, and undo history.
- **scapms::gui:** contains the main window and seven specialized panel widgets.
- **scapms::utils:** handles input validation, ID creation, and CSV file saving/loading.

The full source code is available on GitHub at
https://github.com/praisel-ekpenyong/COS202_SCAPMS

ii. System features

The application covers all required clinic tasks through dedicated tabs:

- **Patient registration:** register, edit and delete patients. The system automatically creates IDs (like P-0001) and validates names, ages, and phone numbers.
- **Appointment:** Schedule, cancel and delete appointments. Staff can filter by date or status, and late visits are highlighted in red.
- **Priority queue:** Patients check into a waiting line. The queue is normally first-come, first-served, but emergency patients are moved to the front. If someone waits longer than 15 minutes, a warning alert shows on the dashboard.
- **Record search:** Find patients by ID, name, or category. The app chooses the best search method (linear, binary, or recursive) depending on whether the data is sorted, and shows which method ran in the status bar.
- **Record sorting:** Sort patient lists by name, ID number, age, or next appointment date using custom implementations of selection sort or merge sort, selectable via dropdown.
- **Undo:** reverse accidental patient deletions, patient edits and appointment removals, most recent first.
- **Reports:** The system generates four reports (frequently attended patients, pending appointments, category breakdown, and daily summary) that can be viewed in tables or exported directly to plain text files.
- **Data storage:** Patients, appointments and ID counters are saved to and loaded from CSV files.

iii. Class hierarchy

PatientRecord is the abstract base class that holds the data every patient share: ID, full name, age, phone number and patient category. It defines two pure virtual methods, **describe()** and **priority()**, which provide runtime polymorphism. Three concrete classes inherit from this base:

- **GeneralPatient** has a baseline priority of 1.
- **FollowUpPatient** has a medium priority of 2 and adds a next-visit date field.
- **EmergencyPatient** has a high priority of 3 (called first)

Each subclass overrides **describe()** to return details specific to its patient type.

To handle appointment capabilities cleanly, I created an abstract interface called `Appointable`. It defines pure virtual methods including **scheduleAppointment(), cancelAppointment()**, **isScheduled(),** and **currentAppointmentId()**. Each concrete patient subclass implements this interface, keeping scheduling separate from general patient details.

Runtime polymorphism is shown by helper functions like **describePatient()** in the model, **describeThroughBase()** in the GUI, and **callNext()** in ClinicManager, which checks priority() through base-class pointers. The program runs the right version for each patient type automatically without needing to know the specific type ahead of time.

ClinicDatabase uses composition by owning patients and appointments through std::unique_ptr. Each Appointment stores its patient's ID, so one patient can have many appointments, retrieved with appointmentsFor(). ClinicManager coordinates the application and works with SearchEngine, PatientSorter and ReportEngine. Undo action form a second hierarchy under the abstract UndoableAction class.

iv. Data structures used

I chose each data structure to suit the needs of the feature that uses it:

- **std::vector:** stores patients and appointments in ClinicDatabase. It grows as needed and gives the constant-time indexing that binary search and merge sort rely on.
- **std::queue:** holds the IDs of waiting patients in arrival order (FIFO), with a std::map storing each arrival time. **callNext()** empties the queue, picks the highest-priority patient, and puts the others back in their original order.
- **std::stack:** stores undo history in ClinicManager. Before deleting or editing a record, the system takes a snapshot of the current state and pushes it onto the stack. Clicking Undo pops the latest snapshot to reverse the change.
- **Fixed-size array (PatientRecord* [10]):** ClinicDatabase keeps a 10-element array as a quick-access cache of the ten most recently accessed patient records. When staff view or edit a record, that record moves to index 0 and older records shift down. If the array is full, the oldest record is dropped.

v. Searching algorithms used

I wrote all the searches by hand in SearchEngine, without using std::find or std::lower_bound.

- **Linear search (O(n))** checks records one by one. It handles ID and name lookups on unsorted data, and category filtering.
- **Binary search (O(log n))** Halves the search range each step. It runs on a sorted list when looking up records by ID or name.
- **Recursive search (O(n))** Searches for a patient by name using recursion instead of a loop.

The technique depends on the state of the data. ClinicManager increases a revision counter whenever records change. If the sorted list's revision does not match the live database, the search automatically falls back to linear search. The status bar always shows which search technique was used.

vi. Sorting algorithms used

I wrote both sorting algorithms as generic functions in `controller/SortAlgorithms.h` and users select either one from a dropdown.

- **Selection sort** repeatedly finds the smallest remaining item and swaps it into place. It takes $O(n^2)$ time, uses no extra memory, and is not stable.
- **Merge sort** recursively splits the list in half, then merges the sorted halves using one scratch buffer. It runs in O(n log n) time on all inputs, uses O(n) extra memory, and it is stable because preserves the original order of duplicate keys.

Records can be sorted by name, ID number, age or upcoming appointment date. Sorting runs on a copy of pointers, so the database order does not change.

vii. Recursive algorithm used

I used recursion for the name search in SearchEngine, which supports exact and prefix matching. The findByNameRecursive function gathers patient pointers and calls a recursive helper function, recursiveNameScan, starting at index 0.
The helper checks one patient at a time and has two base cases:

- **No match:** If the index reaches the end of the list, it returns nullptr.
- **Match found:** If the patient's name matches the search, it returns that patient pointer.

In the recursive step, the helper calls itself with index + 1 to check the next patient. findByNameRecursive then returns a SearchResult containing the first matching patient, or an empty result if no patient matched.

viii. GUI design

MainWindow has a menu bar, a toolbar, a status bar and a QTabWidget with seven tabs: Dashboard, Patient Records, Appointments, Waiting Queue, Search & Sort, Reports and Administration. I chose these layouts for the following reasons:

- **QVBoxLayout and QHBoxLayout** stack each panel's sections and arrange buttons in rows
- **QFormLayout** suits the patient form and appointment dialog because it lines up each label with its input.
- **QGridLayout** suits the dashboard summary and search controls because they fall naturally into rows and columns.

The program uses Qt signals and slots to respond to button clicks, menu actions, dropdown changes, table clicks, pressing Enter to search, and clearing the search box. Two timers run every 30 seconds to refresh dashboard reminders and monitor patient waiting times.

Other GUI features include:

- **Custom colours:** red for emergency, green for follow-up and slate for general patients, plus row tints in the queue.
- **Dynamic components:** the next-visit date appears only when follow-up is selected.
- **File dialogs:** used for choosing data folders and exporting reports.
- **Keyboard shortcuts:** Ctrl+S (Save), Ctrl+F (Search Tab), Ctrl+Q (Exit) and Ctrl+Z (Undo).
- **Tooltips:** Helpful tips added to all main buttons, including short notes that explain each sorting algorithm.
- **Validation messages:** clear messages for invalid input, and confirmation prompt before deleting a records.

ix. Exception handling approach

I implemented custom exception handling in utils/Exceptions.h by defining three classes derived from std::runtime_error:

- **ValidationError** for invalid input values, bad phone numbers and out of range ages,
- **DuplicateIdError** for repeated patients and appointment IDs
- **FileError** for file problems.

The application validates data before making any changes. In `ClinicManager`, methods like `registerPatient` and `scheduleAppointment` validate names, age limits, phone numbers, and date formats before creating objects. If any validation fails, the method throws a `ValidationError` with a specific message. In the GUI layer, user actions are enclosed in try-catch blocks to display user-friendly warnings instead of the application crash. When loading files, a bad row is skipped and reported in a summary dialog while valid rows still load. A FileError shows a warning, or starts the program empty if it occurs at startup.

x. Algorithms chosen and reasons

I selected two sorting algorithms and three searching algorithms for specific clinic workflows:

Sorting Algorithms

- **Merge sort** is used for clinic reports and larger patient lists because its speed stays consistent at O(n log n). It is also stable, meaning patients with the same age or date keep their original order.
- **Selection sort** is used for small lists because it is simple, needs no extra memory, and makes very few swaps. I also use it to sort the appointment table by date and time.

The Search & Sort screen also shows a short note below the dropdown explaining when to use Selection Sort versus Merge Sort.

Searching Algorithms

- **Linear search:** Chosen for unsorted data. Clinic staff add and edit records throughout the day, so sorting the whole database after every change would be unnecessary. Linear search works directly on unsorted data for lookups and category filters.
- **Binary search** Chosen for fast ID or name lookups after the records have been sorted. The system checks a revision number first to make sure the list has not changed, falling back to linear search if it has changed.
- **Recursive search**: Chosen for exact and prefix name searches to demonstrate recursive problem solving using base cases and self calls instead of a loop

xi. Challenges encountered

During the development and testing of SCAPMS, I ran into several practical programming challenges:

1. Deleting a patient who was inside the 10-slot recent cache caused crashes. The cache held raw pointers, so when the patient was deleted, the cache pointed to freed memory. Clicking that entry from the dashboard triggered a crash.
2. My first Merge Sort implementation had off-by-one errors when splitting list with odd numbers of items, causing infinite recursion. Merging without a dedicated scratch buffer also overwrote uncopied data.
3. Sorting patients by name, switched searches to Binary Search. If a user then added a patient or edited a name, that sorted copy became outdated, causing binary searches to miss.
4. An Undo refreshes every panel, and rebuilding the patient table changed its selection programmatically. This fired Qt's itemSelectionChanged signal, which ran the handler meant for user clicks, reloading the form and switching it into edit mode.

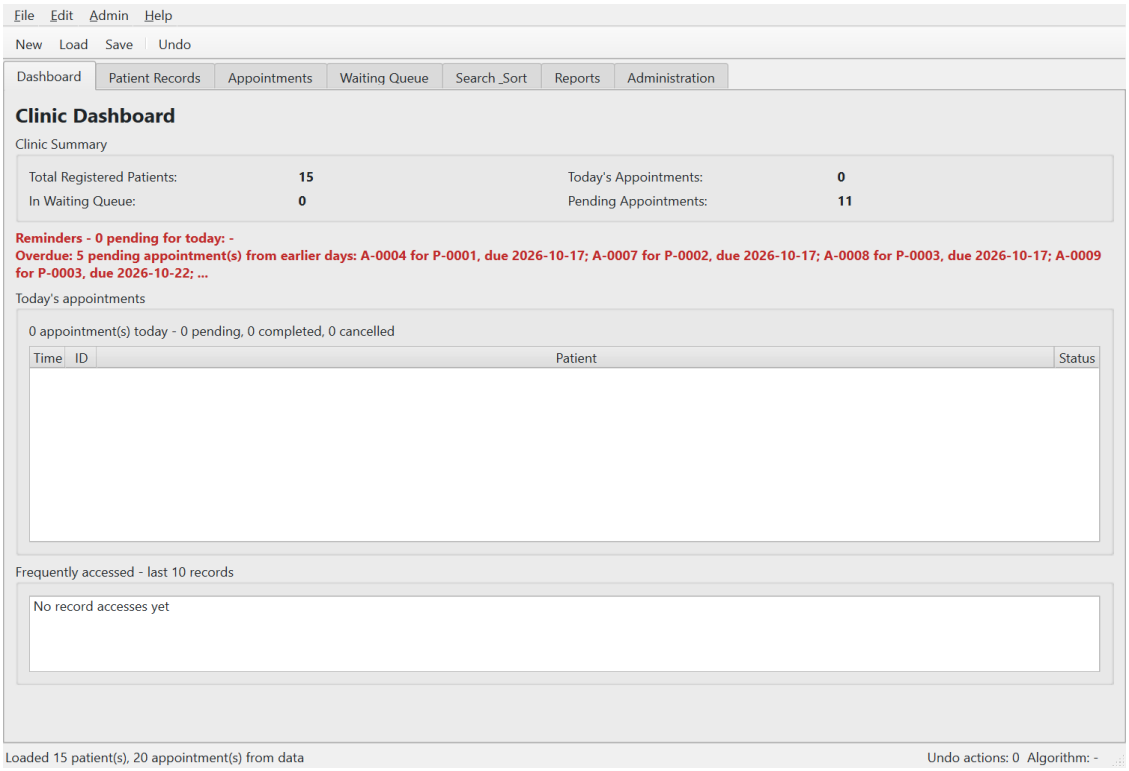
xii. How the challenges were resolved

I resolved these issues with targeted fixes:

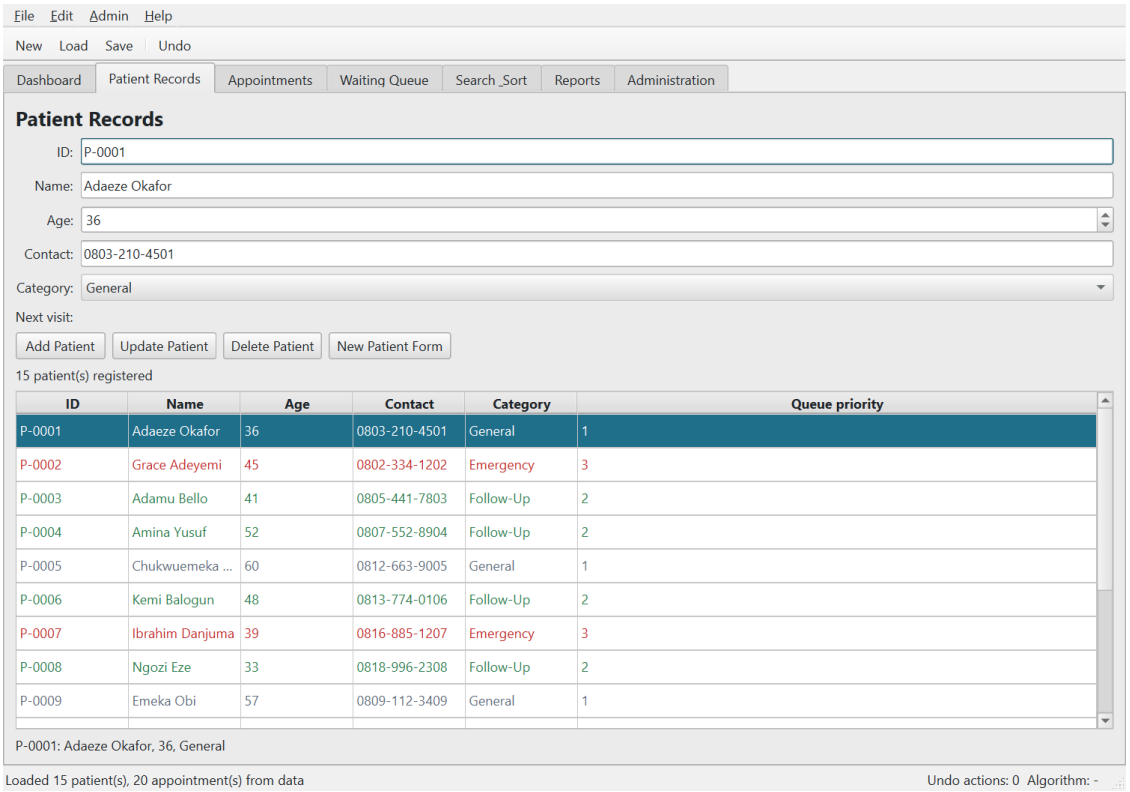
1. To resolve the dangling pointer crash in the cache, I added a `forgetRecent(const std::string& patientId)` method to `ClinicDatabase`. Now, Whenever a patient is deleted, the database clears matching pointers from the cache and shifts remaining items forward before deleting the record.
2. To fix the merge sort errors, I restructured the algorithm in SortAlgorithms.h into two functions. mergeSortRange divides the list using half-open ranges, [lo, mid) and [mid, hi), which made the boundaries consistent for odd and even list sizes. mergeRuns merges the sorted halves into a scratch buffer that mergeSort allocates once at the start, which avoids repeated memory allocation.
3. To stop binary search from using outdated data, I added a revision counter to ClinicManager, which increases whenever patient or appointment records change. PatientSorter stores the current revision with each sorted result. Before running binary search, SearchEngine checks that this revision still matches the live one. If it does not, the search falls back to linear search, and the status bar shows the technique used.
4. To stop the unwanted GUI events, I added an updating_ flag to PatientPanel. The flag is set while the panel updates the table or form itself, and the selection handler exits immediately while it is set.
5. I also made sure that undoing a deletion restores the patient's full state. DeletePatientAction saves the patient's details, all their appointments and their queue arrival time before the deletion takes place.

Part 3: Screenshots

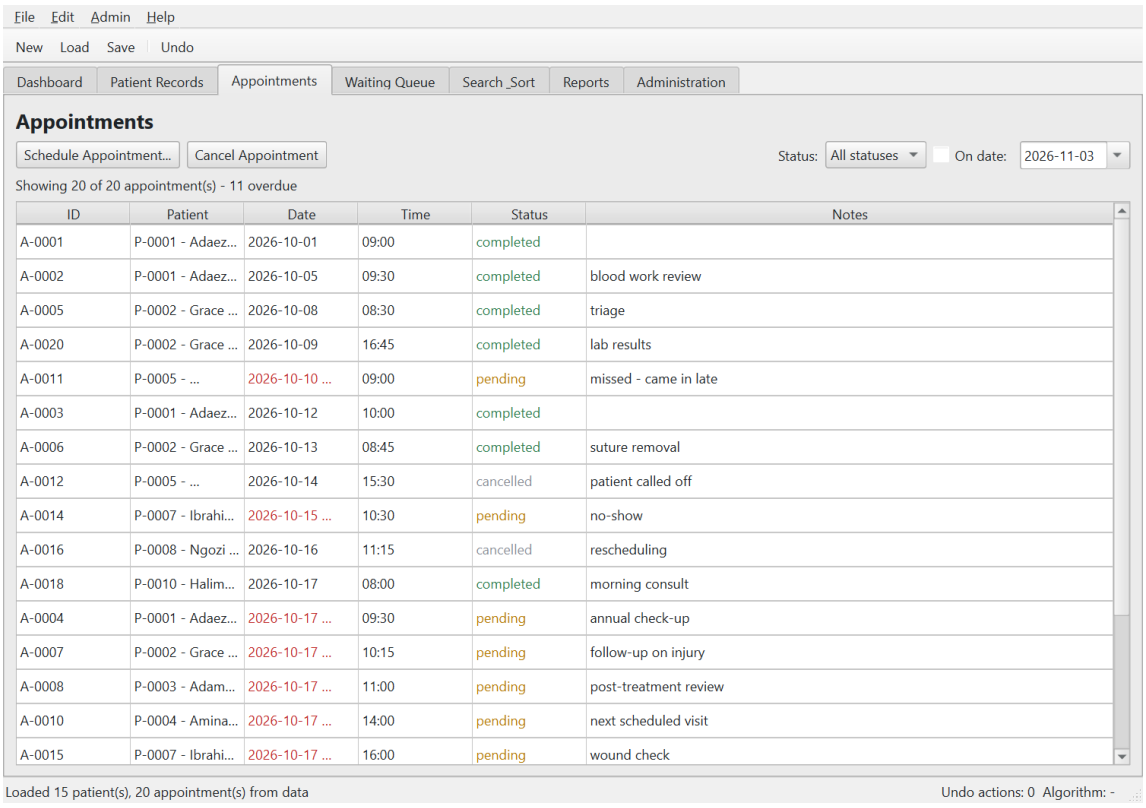
i. Main dashboard



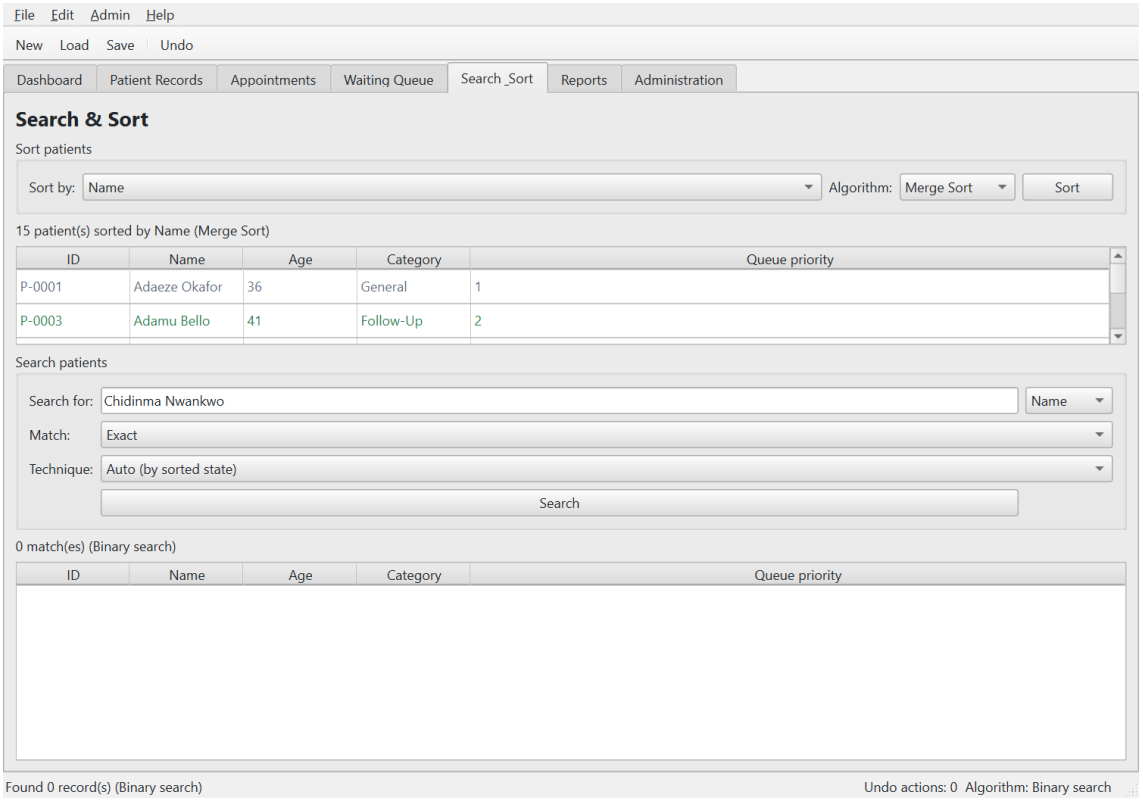
ii. Patient registration



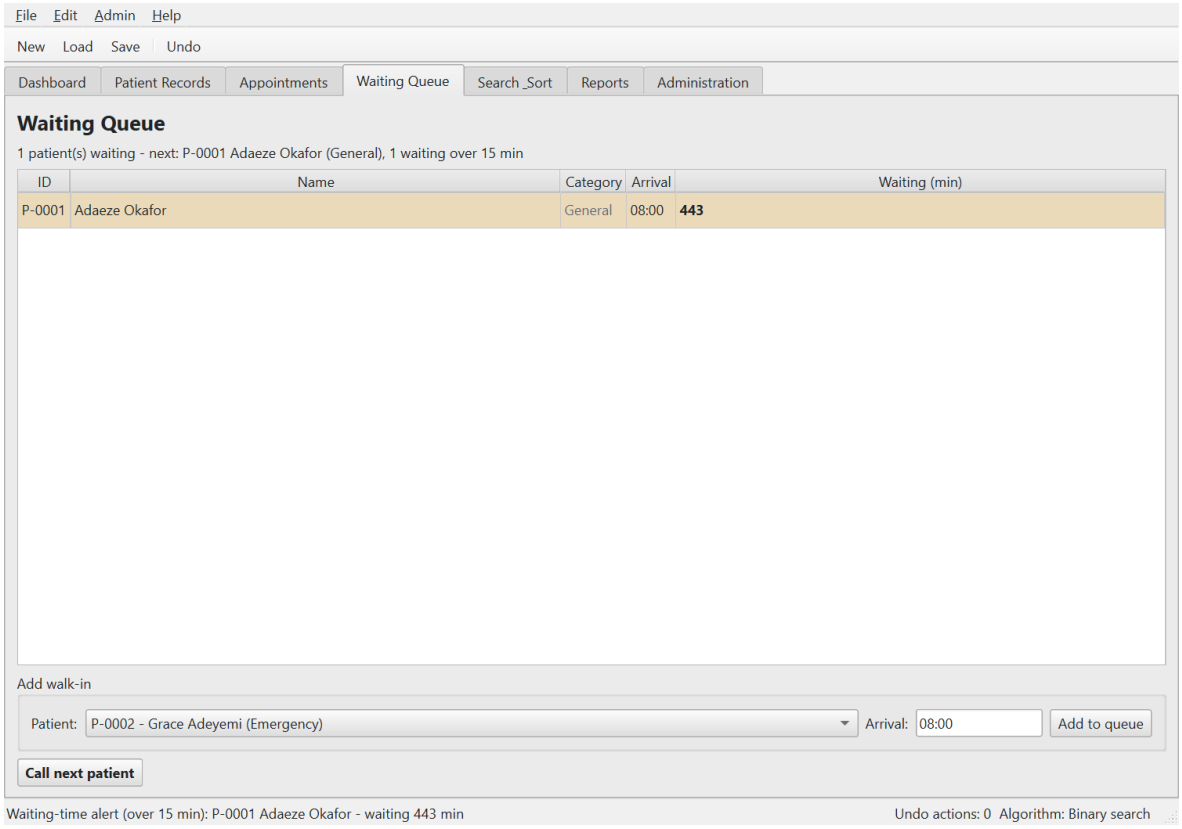
iii. Appointment management



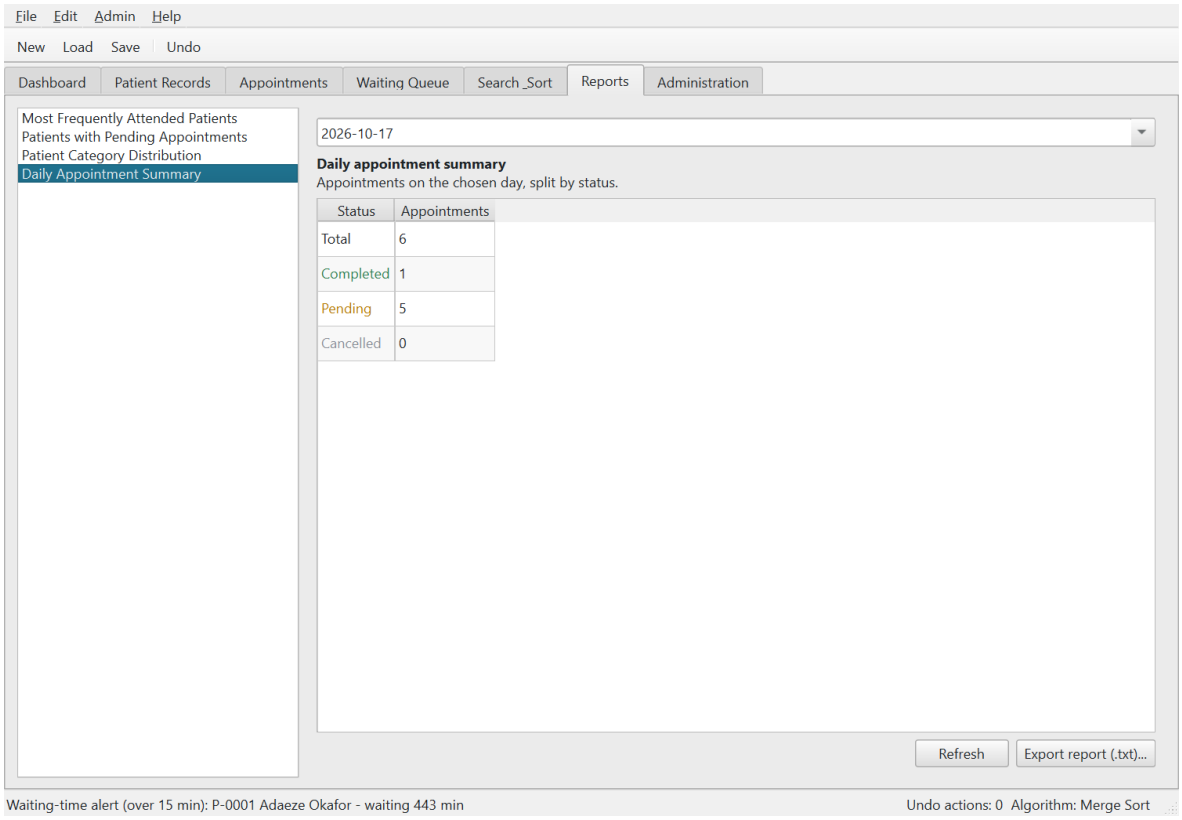
iv. Search/sort interface



v. Waiting queue



vi. Reports



Part 4: UML/Class Diagram – Drawn in draw.io

scapms_class_diagram.drawio

